

Atoms, Literals, Operators & Type Casting

Atoms

- Atoms are the most basic elements of expressions. The simplest atoms are identifiers or literals. Forms enclosed in reverse quotes or in parentheses, brackets or braces are also categorized syntactically as atoms.
- The syntax for atoms is:

atom: identifier | literal |

enclosure: parenth_form | list_display | dict_display | string_conversion

Python keyword

- Keywords are the reserved words in Python.
- We cannot use a keyword as a variable name, function name or any other identifier. They are used to define the syntax and structure of the Python language.
- In Python, keywords are case sensitive.
- There are 33 keywords in Python 3.7. This number can vary slightly in the course of time.
- All the keywords except True, False and None are in lowercase and they must be written as it is.

Python keyword

Keywords in Python

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Python Identifiers

- An identifier is a name given to entities like class, functions, variables, etc. It helps to differentiate one entity from another.
- **Rules for writing identifiers**
 1. Identifiers can be a combination of letters in lowercase (**a to z**) or uppercase (**A to Z**) or digits (**0 to 9**) or an underscore `_`. Names like `myClass`, `var_1` and `print_this_to_screen`, all are valid example.
 2. An identifier cannot start with a digit. `1variable` is invalid, but `variable1` is perfectly fine.
 3. Keywords cannot be used as identifiers.
 4. We cannot use special symbols like `!`, `@`, `#`, `$`, `%` etc. in our identifier.
 5. Identifier can be of any length.

Python Literals

1. **String literals:** String literals can be formed by enclosing a text in the quotes. We can use both single as well as double quotes for a String. **Eg:** "Aman" , '12345'
2. **Numeric literals:** Numeric Literals are immutable. Numeric literals can be signed **integers, long integers, floating point and complex.** **Eg** 100, 87032845L, -26.2, 3.14j
3. **Boolean literals:** A Boolean literal can have any of the two values: True or False.
4. **Special literals.** Python contains one special literal i.e., None. None is used to specify to that field that is not created. It is also used for end of lists in Python.
5. **Literal Collections.** Collections such as tuples, lists and Dictionary are used in Python.

Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
//	Floor division
**	Exponent

- The desired order in which the operations should be performed is unclear but can be clarified by adding parentheses like this:

$$a = (b * c) - ((d \% e) / f)$$

Script to demonstrate arithmetic operators

```
a = 8
b = 2
print( 'Addition:\t' , a , '+' , b , '=' , a + b )
print( 'Subtraction:\t' , a , '-' , b , '=' , a - b )
print( 'Multiplication:\t' , a , 'x' , b , '=' , a * b )
print( 'Division:\t' , a , '/' , b , '=' , a / b )
print( 'Floor Division:\t' , a , '//' , b , '=' , a // b )
print( 'Modulus:\t' , a , '%' , b , '=' , a % b )
print( 'Exponent:\t' , a , '**2' , a ** b , sep = ' ' )
```

NOTE: You can use the **sep** parameter to explicitly specify the separation between output – here it specifies no spaces by assigning two unspaced single quote marks.

Assigning values

Operator:	Example:	Equivalent:
=	a = b	a = b
+=	a += b	a = (a + b)
-=	a -= b	a = (a - b)
*=	a *= b	a = (a * b)
/=	a /= b	a = (a / b)
%=	a %= b	a = (a % b)
//=	a //= b	a = (a // b)
**=	a **= b	a = (a ** b)

Script to demonstrate assignment operators

```
a = 8
b = 4
print( 'Assign Values:\t\t' , 'a =' , a , '\tb =' , b )
a += b
# above is the same as writing a=a+b
print( 'Add & Assign:\t\t' , 'a =' , a , '(8 += 4)' )
a -= b
print( 'Subtract & Assign:\t' , 'a =' , a , '(12 - 4)' )
a *= b
print( 'Multiply & Assign:\t' , 'a =' , a , '(8 * 4)' )
a /= b
print( 'Divide & Assign:\t' , 'a =' , a , '(32 / 4)' )
a %= b
print( 'Modulus & Assign:\t' , 'a =' , a , '(8 % 4)' )
```

NOTE: It is important to regard the = operator to mean “**assign**” rather than “equals” to avoid confusion with the == equality operator.

Comparing values

Operator:	Comparative test:
==	Equality
!=	Inequality
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

- The **== equality operator** compares two operands and will return True if both are equal in value, otherwise it will return a **False** value.
- Conversely, the **!= inequality operator** returns **True** if two operands are not equal, using the same rules as the == equality operator, otherwise it returns **False**.

A-Z uppercase characters have ASCII code values 65-90 and a-z lowercase characters have ASCII code values 97-122.

Script to demonstrate comparison operators

```
nil = 0
```

```
num = 0
```

```
max = 1
```

```
cap = 'A'
```

```
low = 'a'
```

```
print( 'Equality :\t' , nil , '==' , num , nil == num )
```

```
print( 'Equality :\t' , cap , '==' , low , cap == low )
```

```
print( 'Inequality :\t' , nil , '!=' , max , nil != max )
```

```
print( 'Greater :\t' , nil , '>' , max , nil > max )
```

```
print( 'Lesser :\t' , nil , '<' , max , nil < max )
```

```
print( 'More Or Equal :\t' , nil , '>=' , num , nil >= num )
```

```
print( 'Less or Equal :\t' , max , '<=' , num , max <= num )
```

NOTE: The ASCII code value for uppercase “A” is 65 but for lowercase “a” it’s 97 – so their comparison here returns **False**.

Assessing logic

Operator:	Operation:
and	Logical AND
or	Logical OR
not	Logical NOT

- The logical operators are used with operands that have Boolean values of **True or False**, or are values that convert to **True or False**.
- The (logical AND) **and** operator will evaluate two operands and return **True** only if both operands themselves are **True**, otherwise the and operator will return **False**
- The (logical OR) **or** operator will evaluate its two operands and return **True** if either one of the operands itself returns **True**. If neither operand returns True then the or operator will return **False**.
- The (logical NOT) **not** operator is a unary operator that is used before a single operand. It returns the inverse value of the given operand so if the variable a had a value of **True** then not a would have a value of **False**

```
a = True
```

```
b = False
```

```
print( 'AND Logic:' )
```

```
print( 'a and a =' , a and a )
```

```
print( 'a and b =' , a and b )
```

```
print( 'b and b =' , b and b )
```

```
print( '\nOR Logic:' )
```

```
print( 'a or a =' , a or a )
```

```
print( 'a or b =' , a or b )
```

```
print( 'b or b =' , b or b )
```

```
print( '\nNOT Logic:' )
```

```
print( 'a =' , a , '\tnot a =' , not a )
```

```
print( 'b =' , b , '\tnot b =' , not b )
```


Examining conditions

- Unlike other programming languages, Python does not have a **?: ternary operator** but has instead a “conditional expression” that works in a similar way using **if** and **else** keywords with this syntax:

if-true-return-this if (test-expression) else if-false-return-this

Example: `c = a if ( a < b ) else b`

To find greater of the two numbers

```
a = 1
```

```
b = 2
```

```
max = a if ( a > b ) else b
```

```
print( '\nGreater Value Is:' , max )
```

Increment and Decrement Operators

- Increment and Decrement operators (both pre and post) are not allowed.
- Python is designed to be consistent and readable.
- One common error by a novice programmer in languages with ++ and -- operators is mixing up the differences (both in precedence and in return value) between pre and post increment/decrement operators.
- Simple increment and decrement operators aren't needed as much as in other languages.

Setting precedence

Operator:	Description:
**	Exponent
+	Positive
-	Negative
~	Bitwise NOT
*	Multiplication
/	Division
//	Floor division
%	Modulus

Continued..

+

Addition

-

Subtraction

|

Bitwise OR

^

Bitwise XOR

&

Bitwise AND

>>

Bitwise right shift

Continued..

<code><<</code>	Bitwise left shift
<code>>, >=, <, <=, ==, !=</code>	Comparison
<code>=, %=, /=, //=, -=, +=, *=, **=</code>	Assignment
<code>is, is not</code>	Identity
<code>in, not in</code>	Membership
<code>not</code>	Boolean NOT
<code>and</code>	Boolean AND
<code>or</code>	Boolean OR

```
a = 2
```

```
b = 4
```

```
c = 8
```

```
d = a * c + b
```

```
print( '\nDefault Order:\t', a, '*', c, '+', b, '=', d )
```

```
d = a * ( c + b )
```

```
print( 'Forced Order:\t', a, '* (', c, '+', b, ') =', d )
```

```
d = c // b - a
```

```
print( '\nDefault Order:\t', c, '//', b, '-', a, '=', d )
```

```
d = c // ( b - a )
```

```
print( 'Forced Order:\t', c, '// (', b, '-', a, ') =', d )
```

Casting data types

- Although Python variables can store data of any data type it is important to recognize the different types of data they contain to avoid errors when manipulating that data in a program.
- There are several Python data types but by far the most common ones are **str** (string), **int** (integer), and **float** (floating-point).

- Data type recognition is especially important when assigning numeric data to variables from user input as it is stored by default as a **str (string)** data type.
- Fortunately, the data type of stored values can be easily converted (“cast”) into a different data type using built-in Python functions.

NOTE: Converting a **float (floating-point)** data type to an **int (integer)** data type will truncate the number at the decimal point losing the fraction.

Python's built-in data type conversion functions

Function:	Description:
<code>int(x)</code>	Converts <code>x</code> to an integer whole number
<code>float(x)</code>	Converts <code>x</code> to a floating-point number
<code>str(x)</code>	Converts <code>x</code> to a string representation
<code>chr(x)</code>	Converts integer <code>x</code> to a character
<code>unichr(x)</code>	Converts integer <code>x</code> to a Unicode character
<code>ord(x)</code>	Converts character <code>x</code> to its integer value
<code>hex(x)</code>	Converts integer <code>x</code> to a hexadecimal string
<code>oct(x)</code>	Converts integer <code>x</code> to an octal string

Script demonstrating type casting

```
a = input( 'Enter A Number: ' )  
b = input( 'Now Enter Another Number: ' )  
sum = a + b  
print( '\nData Type sum :', sum , type( sum ) )  
sum = int( a ) + int( b )  
print( 'Data Type sum :', sum , type( sum ) )  
sum = float( sum )  
print( 'Data Type sum :', sum , type( sum ) )  
sum = chr( int( sum ) )  
print( 'Data Type sum :', sum , type( sum ) )
```

Manipulating bits

Operator:	Name:	Binary number operation:
	OR	Return a 1 in each bit where either of two compared bits is a 1 Example: 1010 0101 = 1111
&	AND	Return a 1 in each bit where both of two compared bits is a 1 Example: 1010 & 1100 = 1000
~	NOT	Return a 1 in each bit where neither of two compared bits is a 1 Example: 1010 ~ 0011 = 0100
^	XOR	Return a 1 in each bit where only one of two compared bits is a 1 Example: 1010 ^ 0100 = 1110
<<	Shift left	Move each bit that is a 1 a specified number of bits to the left Example: 0010 << 2 = 1000
>>	Shift right	Move each bit that is a 1 a specified number of bits to the right Example: 1000 >> 2 = 0010

```
a = 10
b = 5
print( 'a =', a , '\tb = ' , b )
# 1010 ^ 0101 = 1111 (decimal 15)
a = a ^ b
# 1111 ^ 0101 = 1010 (decimal 10)
b = a ^ b
# 1111 ^ 1010 = 0101 (decimal 5)
a = a ^ b
print( 'a =', a , '\tb = ' , b )
```

NOTE: Do not confuse bitwise operators with logical operators.

Bitwise operators compare binary numbers, whereas logical operators evaluate Boolean values.

Challenges!

- Create a program that asks the user to enter their name and their age. Print out a message addressed to them that tells them the year that they will turn 100 years old.
- Write a Python program which accepts the radius of a circle from the user and compute the area.
- Write a Python program which accepts the user's first and last name and print them in reverse order with a space between them.
- Write a program to check if any 3 digit number is an Armstrong no.
 $153 = \text{cube}(1) + \text{cube}(5) + \text{cube}(3)$
- Write a program to find the largest of 4 Numbers
- Write a program to Check if a Number is Positive Negative or Zero
- Write a program to Check Alphabet
- Write a program to Check if a character is a Vowel

```
num=int(input('Enter the 3 digit no:'))
n1=num%10
num1=num//10
n2=num1%10
num2=num1//10
sum=(n1**3)+(n2**3)+(num2**3)
print('armstrong') if(num==sum) else print('not armstrong')
```

```
ch=input("Enter a lower case alphabet")
```

```
print('Alphabet') if((ch>=chr(65) and ch<=chr(90)) or ch>=chr(97) and ch<=chr(122))
else print('Not alphabet')
```